Looking for GO GSEA methods that support drill down to direct Gene:GO annotations.

It depends a little on what you mean by "drill down to direct Gene:GO annotations" (i.e. you want not just enriched GO terms but a map from your genes → which GO terms drove that enrichment). Below is a list of GO / GSEA / GO-enrichment methods or tools that (or which can be extended to) support that kind of detailed drill-down, plus tips and caveats.

What "drill down to Gene:GO annotations" entails

By that phrase I interpret that you want:

- Given an enriched GO term (or set of GO terms), you want to see *which genes in your input* contributed to that enrichment, via which direct GO annotation(s).
- Optionally, you might also want to consider GO topology (i.e. inheritance / parent/child relationships) and possibly exclude indirect contributions.

Many GO enrichment / GSEA methods report only term-level p-values or enrichment scores, but you can often retrieve, or extend, them to get the per-gene associations.

Candidate methods / tools with good support for drill-down

Here are some GO / GSEA / GO-enrichment methods / toolkits that are good choices for drill-down:

Method / Tool	Strengths re: drill-down	Notes / how to get gene-term mapping
topGO (R / Bioconductor)	Very good. You can extract, for each GO term you test, the set of genes annotated to it via `genesInTerm()`, and then intersect with your "significant" genes. Bioconductor +4	For example, after running `runTest()`, you can `genesInTerm(GOdata, whichGO = <list_of_GO>` get the genes for each GO term. Bioconductor +4
goseq (for RNA-seq bias correction)**	Good, though more focused on overrepresentation. `goseq` has a helper function `getgo()` to fetch gene → GO mapping, which you can use to relate enriched GO terms back to genes. Bioconductor +4	`getgo(genes, genome, id, fetch.cats = c("G "GO:MF", "GO:CC"))` returns for each gene a ve GO terms. Using that mapping, you can intersect enriched GO sets. Bioconductor
clusterProfiler (R / Bioconductor)**	While clusterProfiler is more generic (GO, KEGG, etc.), it gives gene-level membership via the `geneID` column in enrichment result tables. Many users extract the `@core_enrichment` field (a string of gene IDs) which tells you which genes contributed.	It supports GO and GSEA style analyses; you'll g mapping from GO → gene list in the output objec
GSEA / fgsea (for GO gene sets)	GSEA is more a general framework; if you supply GO gene sets (e.g. from MSigDB), then the output (e.g. leading edge genes) gives you which genes in that gene set drove the enrichment.	The "leading edge subset" is often a way to focu core genes driving the GO set enrichment.
g:Profiler	g:Profiler returns for each enriched GO term the mapped genes from your input (i.e. which of your genes map to it). biit.cs.ut.ee	You can fetch a detailed table of gene → term ma in the output.

Method / Tool	Strengths re: drill-down	Notes / how to get gene-term mapping
Enrichr	Enrichr returns gene lists for each enriched GO term in its results. maayanlab.cloud	The web UI / API shows which genes in your list r each GO term.

Practical suggestions: how to combine enrichment + drill-down

- **Use a method that retains gene membership**: When choosing a GO-enrichment method, prefer ones that either natively store or report which genes contributed to each enriched term (e.g. topGO's `genesInTerm`, clusterProfiler's `core_enrichment`, Enrichr gene lists).
- **Post-hoc mapping**: Even if a tool doesn't directly give you gene-term membership, you can always export your enriched GO terms and then use a GO annotation database (e.g. GOA, Bioconductor annotation packages, or `org.*.db`) to map your input gene list to GO terms, and thus reconstruct the gene → GO relationships.
- **Filtering / conditional analysis**: Because of GO's hierarchy, sometimes a "parent" GO term appears enriched just because one of its children is doing so. Methods like topGO support "elim" / "weight" / "parentChild" algorithms to help reduce this effect. Bioconductor +2
- **Leading edge analysis**: In GSEA style implementations, the concept of the "leading edge" (the subset of genes contributing most to the enrichment signal) is often a natural way to drill down into core contributing genes.

If you like, I can prepare a short worked example (in R or Python) showing GO enrichment + gene drill-down (e.g. with topGO or clusterProfiler) for your kind of data. Do you want me to do that (and in which language)?

Sources

Prefer API call or Python package (R not ideal)

Got it — you'd prefer something you can call programmatically from **Python** or via an **API**, rather than R packages. That narrows the field.

Here are the main options you can use in **Python** for GO GSEA / enrichment *with gene-level drill down*:

🔧 Python Packages

1. GSEApy

- Python wrapper for Enrichr, GSEA, and g:Profiler.
- Supports:
  - **Overrepresentation analysis** (ORA) and **GSEA** using GO terms from Enrichr/MSigDB.
  - Returns enrichment results **with the contributing genes** (`genes` column).
  - Also exposes the **leading edge genes** for GSEA.
- Example:

python

```python
import gseapy as gp

enr = gp.enrichr(
    gene_list=['TP53','BRCA1','MYC'],
    gene_sets='GO_Biological_Process_2023',
    organism='Human'
)

# Results include a 'Genes' column with contributing genes
print(enr.results[['Term','Genes']])
```

- Docs: https://gseapy.readthedocs.io ↗

---

2. GOATOOLS

- Python library focused on **GO annotation and enrichment**.
- Lets you load a GO ontology file (`go-basic.obo`) and a GAF (gene association file).
- Provides enrichment tests (Fisher, etc.) and you can easily extract which **genes map to which GO term**.
- Example:

```python
from goatools.obo_parser import GODag
from goatools.goea.go_enrichment_ns import GOEnrichmentStudy

# load GO ontology
go_dag = GODag("go-basic.obo")

# load associations: dict {gene: [go_terms]}
assoc = {'gene1':['GO:0008150'], 'gene2':['GO:0009987']}

study_genes = ['gene1','gene2']
population = ['gene1','gene2','gene3']

goeaobj = GOEnrichmentStudy(
    population, assoc, go_dag,
    methods=['fdr_bh']
)
results = goeaobj.run_study(study_genes)

for r in results:
    if r.p_fdr_bh < 0.05:
        print(r.GO, r.name, r.study_items)  # study_items = genes cont
```

- Very flexible if you want **direct gene→GO drill down**.
- Docs: https://github.com/tanghaibao/goatools ↗

🌐 APIs with Python clients

3. g:Profiler API

- Official API with Python client (`gprofiler-official`).
- Returns enriched GO terms **plus the input genes mapped to each term**.
- Example:

```python
```

```
from gprofiler import GProfiler

gp = GProfiler(return_dataframe=True)
res = gp.profile(organism='hsapiens', query=['TP53','BRCA1','MYC'])

print(res[['name','intersections']])  # intersections = contributing g
```

- Docs: https://pypi.org/project/gprofiler-official/ ↗

---

4. Enrichr API

- Simple HTTP API, also wrapped by GSEApy.
- Returns enriched GO terms with lists of contributing genes.
- Example with requests:

python

```
import requests

url = "https://maayanlab.cloud/Enrichr/addList"
genes = "\n".join(["TP53","BRCA1","MYC"])
payload = {'list': (None, genes)}

res = requests.post(url, files=payload).json()
user_list_id = res['userListId']

enrich_url = f"https://maayanlab.cloud/Enrichr/enrich?userListId={user_
enrich_res = requests.get(enrich_url).json()

# Each enrichment entry includes genes contributing
```

- Docs: https://maayanlab.cloud/Enrichr/help ↗

✅ **Summary**:

- If you want **pure Python, offline, and full control** → **GOATOOLS**.
- If you want **convenience + multiple gene set databases** → **GSEApy** (uses Enrichr/GSEA/g:Profiler).
- If you want **official well-maintained API** → **g:Profiler API**.
- If you want **simple cloud-based enrichment** → **Enrichr API**.

Copy code

👉 Do you want me to sketch a **ready-to-run Python workflow** (load gene list → run enrichment → return GO term + contributing genes) using one of these (say GSEApy or GOATOOLS)?

This is a correct description of what I'm aiming for
> you want not just enriched GO terms but a map from your genes → which GO terms drove that enrichment
In GO enrichment there is a link between GO term in enrichment results and genes annotated to it directly or indirectly. Then there is the direct annotation of those

genes to terms under the GO term in the enrichment results. I'd really like both of these.

Perfect — thanks for clarifying. You want two different (but related) levels of drill-down for enriched GO terms:

1. **All contributing genes**:
   - The usual enrichment output → GO term enriched, and the *set of your genes that map to it*.
   - This mapping typically includes **direct + inherited (propagated)** annotations (because if a gene is annotated to a child, it counts for the parent too).
2. **Direct annotations only**:
   - For each contributing gene, you want to know if the gene has a **direct annotation** to that enriched GO term (from the GAF file) or if it's only contributing via a descendant term.

That's a subtle but important distinction — many tools stop at (1), but you want both (1) and (2).

Which Python tools can give you both?

GOATOOLS

- This is the strongest option for your needs because:
  - It loads the **full GO DAG (ontology)**.
  - You can parse a **GAF file** (which contains direct gene→term annotations).
  - It automatically expands to parents for enrichment (so you get the indirect mapping).
  - But since you also have the raw GAF, you can distinguish between "direct" and "indirect" contributions.

Concretely:

- `association` (from GAF) → gives you **direct** gene→GO mappings.
- Enrichment study results (`r.study_items`) → gives you the **genes contributing to the enriched term**, which may include indirect annotations.
- You can compare the two:
  - Genes in `r.study_items` ∩ `assoc[gene]` = direct.
  - Genes in `r.study_items` but not directly annotated = indirect (via descendants).

---

g:Profiler / Enrichr / GSEApy

- These APIs return enriched terms with the **list of contributing genes**.
- But they do not expose whether the annotation is **direct or via child term** — because they work on pre-expanded gene sets.
- So they're excellent for the "all contributing genes" view but not the direct vs. indirect split.

🛠️ Workflow in Python with GOATOOLS

Here's a sketch of how you'd do both levels:

python Copy code

```python
from goatools.obo_parser import GODag
from goatools.associations import read_gaf
from goatools.goea.go_enrichment_ns import GOEnrichmentStudy

# Load GO DAG
go_dag = GODag("go-basic.obo")

# Load direct gene-to-GO annotations from a GAF file (e.g. from Gene Ontol
assoc = read_gaf("gene_association.gaf")

# Define your population and study genes
population = list(assoc.keys())  # all genes in GAF
study_genes = ['TP53', 'BRCA1', 'MYC']  # your gene list

# Run enrichment
goea = GOEnrichmentStudy(population, assoc, go_dag, methods=['fdr_bh'])
results = goea.run_study(study_genes)

# Inspect enriched terms
for r in results:
    if r.p_fdr_bh < 0.05:
        print(f"GO term: {r.GO} {r.name}")
        print("  All contributing genes:", r.study_items)

        # Directly annotated genes: intersection with original assoc
        direct_genes = [g for g in r.study_items if r.GO in assoc.get(g, [
        print("  Direct annotations:", direct_genes)

        # Indirect = contributing genes but not directly annotated
        indirect_genes = set(r.study_items) - set(direct_genes)
        print("  Indirect via descendants:", indirect_genes)
```

This gives you, for each enriched GO term:

- **All contributing genes** (direct + indirect).
- **Directly annotated genes**.
- **Indirect contributors (via child terms)**.

✅ So in practice:

- **GOATOOLS** is your best option if you want *both direct + indirect mapping*.
- **g:Profiler/Enrichr/GSEApy** are great if you only need "which genes contributed" but don't care about direct vs. propagated annotation.

Would you like me to put together a **fully runnable Python notebook** (using GOATOOLS, GO DAG + a GAF, small toy gene list) that outputs exactly these 3 columns (term → all genes, direct genes, indirect genes)?

So I can use GOATOOLS in python?

Yes ✅ — **GOATOOLS** is a native Python package, actively maintained, and it's exactly designed for this kind of GO analysis.

With it you can:

- Parse the **GO ontology** (`go-basic.obo`) → gives you the DAG structure.
- Parse **annotation files** (GAFs from Gene Ontology or UniProt) → gives you direct gene→term relationships.
- Run **GO enrichment** → it expands those direct annotations up the DAG so you get both direct and inherited contributions.
- Inspect results at both levels:
  - `r.study_items` = all genes contributing to an enriched GO term (direct + indirect).
  - You can cross-check with the raw GAF associations to see which are *direct* vs. *only via child terms*.

🔧 Install

bash Copy code

```bash
pip install goatools
```

🔧 Minimal Example

python Copy code

```python
from goatools.obo_parser import GODag
from goatools.associations import read_gaf
from goatools.goea.go_enrichment_ns import GOEnrichmentStudy

# Load GO DAG (ontology graph)
go_dag = GODag("go-basic.obo")

# Load GAF with gene-to-GO direct annotations
assoc = read_gaf("gene_association.gaf")

# Background = all annotated genes
population = list(assoc.keys())

# Your study genes
study_genes = ['TP53', 'BRCA1', 'MYC']

# Run enrichment
goea = GOEnrichmentStudy(population, assoc, go_dag, methods=['fdr_bh'])
results = goea.run_study(study_genes)

# Drill-down: enriched terms
for r in results:
    if r.p_fdr_bh < 0.05:
        print(f"\nGO term: {r.GO} {r.name}")

        # All contributing genes (direct + indirect)
        print("  All:", r.study_items)

        # Directly annotated genes
        direct = [g for g in r.study_items if r.GO in assoc.get(g, [])]
        print("  Direct:", direct)

        # Indirect contributors via children
```

```
        indirect = set(r.study_items) - set(direct)
        print("  Indirect:", indirect)
```

So yes: **GOATOOLS in Python** can give you exactly the split you want:

- All genes contributing to an enriched GO term
- Which are **directly annotated**
- Which are **indirect via children**

👉 Do you want me to show you how to fetch a **real GAF + OBO** from Gene Ontology (instead of placeholders) so you can run this out-of-the-box?

yes please

Great 👍 — let's make this fully runnable with **real GO files**.

---

1. Download the required GO resources

You need two files:

1. **GO ontology (DAG structure)** → `go-basic.obo`
   - URL: http://purl.obolibrary.org/obo/go/go-basic.obo ↗
   - This is the standard DAG used for enrichment.
2. **Gene annotations (GAF file)** → for your organism
   - These contain **direct annotations** (gene → GO term).
   - Examples:
     - **Human (UniProt)**: http://current.geneontology.org/annotations/goa_human.gaf.gz ↗
     - **Mouse**: http://current.geneontology.org/annotations/mgi.gaf.gz ↗
     - Full list: http://current.geneontology.org/annotations/ ↗

---

2. Python example with GOATOOLS

python Copy code

```python
import gzip
from goatools.obo_parser import GODag
from goatools.associations import read_gaf
from goatools.goea.go_enrichment_ns import GOEnrichmentStudy

# ----------------------------
# STEP 1: Load ontology (DAG)
# ----------------------------
go_dag = GODag("go-basic.obo")

# ----------------------------
# STEP 2: Load direct gene->GO associations from GAF
# ----------------------------
# If file is gzipped, open with gzip
```

```
with gzip.open("goa_human.gaf.gz", "rt") as gaf:
    assoc = read_gaf(gaf)

print(f"Loaded {len(assoc)} genes with direct GO annotations")

# ----------------------------
# STEP 3: Define background & study genes
# ----------------------------
population = list(assoc.keys())  # all annotated human genes

# Example query genes (real gene symbols must match the GAF IDs, e.g. UniP
study_genes = ["P04637", "P38398", "Q00987"]  # TP53, BRCA1, MDM2 UniProt

# ----------------------------
# STEP 4: Run enrichment
# ----------------------------
goea = GOEnrichmentStudy(
    population,
    assoc,
    go_dag,
    propagate_counts=True,  # count child->parent
    alpha=0.05,
    methods=['fdr_bh']
)

results = goea.run_study(study_genes)

# ----------------------------
# STEP 5: Inspect enriched terms
# ----------------------------
for r in results:
    if r.p_fdr_bh < 0.05:
        print(f"\nGO term: {r.GO} {r.name}  p={r.p_fdr_bh:.3e}")

        # All contributing genes (direct + indirect)
        print("  All contributing genes:", r.study_items)

        # Directly annotated genes
        direct = [g for g in r.study_items if r.GO in assoc.get(g, [])]
        print("  Direct annotations:", direct)

        # Indirect contributors (via child terms)
        indirect = set(r.study_items) - set(direct)
        print("  Indirect via descendants:", indirect)
```

---

3. Notes

- **Gene IDs** in GAFs:
  - Human `goa_human.gaf` uses **UniProtKB IDs** (e.g. `P04637` for TP53).
  - You'll need to map your gene symbols → UniProt IDs before running enrichment.
- You can use mygene ↗ Python client or UniProt mapping API for conversion.
- Other organisms (mouse, yeast, fly, etc.) have organism-specific GAFs.

✅ This setup gives you exactly what you asked for:

- Enriched GO terms

- Contributing genes
- Split into **direct annotations** vs. **indirect (via descendants)**

Would you like me to also show you a **small helper function** that takes gene symbols (like `TP53`, `BRCA1`) and auto-converts them to UniProt IDs before running GOATOOLS?

...: print(f"Loaded {len(assoc)} genes with direct GO annotations")
Traceback (most recent call last):
  File "/Users/do12/Documents/GitHub/GO_enrichment_scratch/.venv/lib/python3.12/site-packages/goatools/anno/init/reader_gaf.py", line 66, in _read_gaf_nts
    with open(self.fin_gaf) as ifstrm:
         ^^^^^^^^^^^^^^^^^^
TypeError: expected str, bytes or os.PathLike object, not TextIOWrapper

**FATAL-gaf: expected str, bytes or os.PathLike object, not TextIOWrapper

**FATAL-gaf: <_io.TextIOWrapper name='goa_human.gaf.gz' encoding='UTF-8'>[-1]:
An exception has occurred, use %tb to see the full traceback.

SystemExit: 1

Ah 👍 I see the issue.

`read_gaf` in **GOATOOLS** expects either:

- a **filename** (string path), even if it's gzipped (it auto-detects `.gz`)
- or a **file-like iterator** of parsed GAF lines, but not a raw `TextIOWrapper`.

In the code I gave you, I wrapped the gzipped file in `gzip.open(..., "rt")` and passed the handle — that's why you got the `TypeError`.

✅ Two correct ways to load the GAF

Option 1 (simplest): just pass the filename

GOATOOLS can handle `.gz` files directly:

python Copy code

```python
from goatools.associations import read_gaf

assoc = read_gaf("goa_human.gaf.gz")  # just give the path, works with gz
print(f"Loaded {len(assoc)} genes with direct GO annotations")
```

Option 2: manual stream

If you *really* want to open the file yourself, then you should pass an **iterator of strings**, not the file object:

python Copy code

```python
import gzip
from goatools.associations import read_gaf

with gzip.open("goa_human.gaf.gz", "rt") as f:
    assoc = read_gaf(f.readlines())  # pass list of lines
```

👉 I recommend **Option 1**, since GOATOOLS does the `.gz` decompression automatically.

Would you like me to also show you how to **map gene symbols (e.g. TP53)** to the UniProt IDs in the `goa_human.gaf` automatically, so you don't have to manually look up IDs?

In [29]: from goatools.associations import read_gaf
...:
...: assoc = read_gaf("goa_human.gaf.gz") # just give the path, works with gz
...: print(f"Loaded {len(assoc)} genes with direct GO annotations")
Traceback (most recent call last):
File "/Users/do12/Documents/GitHub/GO_enrichment_scratch/.venv/lib/python3.12/site-packages/goatools/anno/init/reader_gaf.py", line 67, in _read_gaf_nts
for lnum, line in enumerate(ifstrm, 1):
^^^^^^^^^^^^^^^^^^^^
File "<frozen codecs>", line 322, in decode
UnicodeDecodeError: 'utf-8' codec can't decode byte 0x8b in position 1: invalid start byte

**FATAL-gaf: 'utf-8' codec can't decode byte 0x8b in position 1: invalid start byte

**FATAL-gaf: goa_human.gaf.gz[-1]:
An exception has occurred, use %tb to see the full traceback.

Got it — thanks for pasting the traceback 🙏

The error `UnicodeDecodeError: 'utf-8' codec can't decode byte 0x8b...` means that **GOATOOLS is trying to read the gzipped file as plain text**.

Unlike what I said earlier, the current `read_gaf` in GOATOOLS **does not auto-detect** `.gz`. You need to decompress the `.gz` yourself and feed the plain text.

✅ Correct way to read `goa_human.gaf.gz`

python Copy code

```python
import gzip
from goatools.associations import read_gaf

with gzip.open("goa_human.gaf.gz", "rt") as f:  # "rt" = text mode
    assoc = read_gaf(f)

print(f"Loaded {len(assoc)} genes with direct GO annotations")
```

This works because:

- `gzip.open(..., "rt")` produces a text stream (decoded line by line).
- `read_gaf` accepts a file-like iterator of lines.

🔎 Quick sanity check

After loading, you can confirm a few things:

python Copy code

```python
print(len(assoc))              # number of genes with annotations
some_gene = next(iter(assoc))
print("Example gene:", some_gene)
print("Direct GO terms:", assoc[some_gene])
```

👉 So the rule is: **uncompress first (via gzip or manually), then pass the handle to** `read_gaf`.

Would you like me to also show you how to **map gene symbols like** `TP53` **→ UniProt IDs** (so you don't need to work with `P04637` etc. directly in your gene list)?

The report I want: GO term with significant enrichment (set high P value) --> Gene --> terms that gene is directly annotated to.

Perfect — that's very clear now ✅

You want a **report structured like this**:

vbnet Copy code

```vbnet
GO:0006950 — response to stress (p=0.02)
  TP53 → [GO:0006977 DNA damage response, GO:0043524 apoptotic program]
  BRCA1 → [GO:0006281 DNA repair]
  MDM2  → [GO:0042803 protein homodimerization]
```

That is:

- **GO enrichment result** (enriched term, with p-value).

- **For each gene contributing to that enrichment** → list of the **direct annotations** that gene has (from the GAF), *not just the parent term itself*.

How to generate this with GOATOOLS

Steps:

1. Load ontology (OBO) + GAF annotations.
2. Run enrichment.
3. For each enriched GO term:
   - Get the **study genes contributing** (`r.study_items`).
   - For each such gene, list its **direct annotations** (`assoc[gene]`) that are **children/descendants of the enriched term** (or possibly equal to the term).

That way you see *why* the gene is contributing (i.e. what direct evidence underlies the indirect parent enrichment).

Example Python Code

python Copy code

```python
import gzip
from collections import defaultdict
from goatools.obo_parser import GODag
from goatools.associations import read_gaf
from goatools.goea.go_enrichment_ns import GOEnrichmentStudy

# Load ontology
go_dag = GODag("go-basic.obo")

# Load direct gene->GO associations (from GAF)
with gzip.open("goa_human.gaf.gz", "rt") as f:
    assoc = read_gaf(f)

population = list(assoc.keys())
study_genes = ["P04637", "P38398", "Q00987"]  # TP53, BRCA1, MDM2 (UniProt

# Run enrichment with a very permissive cutoff
goea = GOEnrichmentStudy(population, assoc, go_dag, propagate_counts=True,
results = goea.run_study(study_genes)

# Build the report
for r in results:
    if r.p_fdr_bh < 0.1:  # set high cutoff
        print(f"\n{r.GO} — {r.name}  (p={r.p_fdr_bh:.3e})")
        for g in r.study_items:
            # Find direct GO terms of gene that fall under this enriched t
            direct_terms = assoc.get(g, [])
            relevant = [t for t in direct_terms if t == r.GO or t in go_da
            if relevant:
                names = [f"{tid} {go_dag[tid].name}" for tid in relevant]
                print(f"  {g} → {names}")
```

What you'll get

- Each **significant GO term**.
- Each **contributing gene**.

- The **set of GO terms that gene is directly annotated to** which "explain" its contribution.

This report answers exactly:
👉 *Which direct annotations are causing each gene to appear under a higher-level enriched GO term?*

⚡ Do you want me to extend this so that instead of UniProt IDs (`P04637`) you automatically see **gene symbols** (TP53, BRCA1, etc.) in the report?

for some reason go_dag is empty. loaded using `go_dag = GODag("go-basic.obo")` The file is on disk - downloaded using WGET

Ah — if your `GODag("go-basic.obo")` call produced an *empty DAG*, that usually means the file didn't download correctly or got truncated. `wget` sometimes saves a **redirect page (HTML)** instead of the real `.obo` .

✅ Things to check

1. **Check file size**

   bash Copy code

   ```bash
   ls -lh go-basic.obo
   ```

   - A valid `go-basic.obo` should be ~30–40 MB (as of 2025).
   - If it's only a few KB → you probably got an HTML error page.

2. **Check first few lines**

   bash Copy code

   ```bash
   head go-basic.obo
   ```

   - Should look like:

     makefile Copy code

     ```makefile
     format-version: 1.2
     data-version: releases/YYYY-MM-DD
     subsetdef: goslim_yeast ...
     [Term]
     id: GO:0000001
     name: mitochondrion inheritance
     ...
     ```

   - If you see HTML → wrong file.

✅ Correct download command

Use the canonical PURL (redirects to the latest file):

bash Copy code

```
wget http://purl.obolibrary.org/obo/go/go-basic.obo
```

Or with curl:

bash Copy code

```bash
curl -L -O http://purl.obolibrary.org/obo/go/go-basic.obo
```

(`-L` follows redirects, which wget usually does, but sometimes not in strict environments.)

✅ Reload in Python

python Copy code

```python
from goatools.obo_parser import GODag

go_dag = GODag("go-basic.obo")
print(f"Loaded {len(go_dag)} GO terms from ontology")
```

Expected: ~45,000 terms.

⚡ Quick question: when you ran `head go-basic.obo`, did you see OBO header lines or just HTML? That will tell us if your file is corrupted or if there's a parsing problem. Would you like me to show you a one-liner in Python that **checks for HTML vs OBO** in your file?

```
Calculating 14,626 uncorrected p-values using
fisher_scipy_stats
  14,626 terms are associated with 36,540 of 36,540
population items
    760 terms are associated with      3 of      3 study items
---------------------------------------------------------
------------------
AssertionError                            Traceback (most recent
call last)
Cell In[54], line 18
    16 # Run enrichment with a very permissive cutoff
    17 goea = GOEnrichmentStudy(population, assoc,
go_dag, propagate_counts=True, alpha=1.0, methods=
['fdr_bh'])
---> 18 results = goea.run_study(study_genes)
    20 # Build the report
    21 for r in results:

File
~/Documents/GitHub/GO_enrichment_scratch/.venv/lib/p
ython3.12/site-
packages/goatools/go_enrichment.py:350, in
GOEnrichmentStudy.run_study(self, study, **kws)
   343     log.write(
   344         "  {MSG}\n".format(
   345             MSG="\n
".join(self.get_results_msg(results, study))
```

```
    346         )
    347     )
    349 # Do multipletest corrections on uncorrected
pvalues and update results
--> 350 self._run_multitest_corr(results, methods, alpha,
study, log)
    352 for rec in results:
    353     # get go term for name and level
    354     rec.set_goterm(self.obo_dag)

File
~/Documents/GitHub/GO_enrichment_scratch/.venv/lib/p
ython3.12/site-
packages/goatools/go_enrichment.py:448, in
GOEnrichmentStudy._run_multitest_corr(self, results,
usrmethod_flds, alpha, study, log)
    446 def _run_multitest_corr(self, results,
usrmethod_flds, alpha, study, log):
    447     """Do multiple-test corrections on uncorrected
pvalues."""
--> 448     assert 0 < alpha < 1, "Test-wise alpha must fall
between (0, 1)"
    449     pvals = [r.p_uncorrected for r in results]
    450     ntobj = cx.namedtuple("ntobj", "results pvals
alpha nt_method study")

AssertionError: Test-wise alpha must fall between (0, 1)
```

Aha — that traceback tells the story.

The error comes from this line:

```python
goea = GOEnrichmentStudy(population, assoc, go_dag,
                         propagate_counts=True, alpha=1.0, methods=['fdr_b
```

Why it failed

- In GOATOOLS, the `alpha` parameter must be **strictly between 0 and 1** (`0 < alpha < 1`).
- You set `alpha=1.0` , which triggered the assertion:

```java
AssertionError: Test-wise alpha must fall between (0, 1)
```

✅ Fix

If you want to keep **all terms** and just filter later, set `alpha` to something close to 1 but still < 1, e.g. `0.999999` .
Or set it to a high value like `0.99` :

```python
goea = GOEnrichmentStudy(population, assoc, go_dag,
```

```
propagate_counts=True, alpha=0.99, methods=['fdr_
```

Then when you iterate over results, you can manually apply whatever cutoff you want:

python Copy code

```python
results = goea.run_study(study_genes)

for r in results:
    if r.p_fdr_bh < 0.5:  # or whatever high threshold you want
        print(r.GO, r.name, r.p_fdr_bh, r.study_items)
```

⚡ So the rule is:

- `alpha` must be `(0,1)` — it's used internally for the multiple-testing procedure.
- To "set a high cutoff," you don't make `alpha=1.0`; instead you set `alpha=0.99` (or lower) and then filter the `results` yourself however you like.

👉 Do you want me to show you how to wrap this into a **helper function** that directly outputs your desired report structure (GO term → gene → direct annotations), with a configurable cutoff, so you don't need to worry about `alpha`?

What would be considered a reasonable, strict cuttoff for significance?

Good question 👍 — the "right" cutoff depends on how conservative you want to be, and how you plan to use the results.

In practice for **GO enrichment / GSEA** people usually report both:

🔬 Common thresholds

- **Uncorrected p-value**:
  - Not meaningful by itself if you test thousands of terms → always apply multiple test correction.
- **FDR-adjusted p-value (q-value)**:
  - **FDR < 0.05** (5%) → the most common threshold.
  - **FDR < 0.01** → stricter, often used in high-confidence studies.
  - Some groups go as high as **FDR < 0.10** if they want to capture broader biology and are aware of more false positives.
- **Bonferroni correction**:
  - Very strict, often too conservative for GO (~10–40k terms tested).
  - Usually not recommended unless you absolutely need to eliminate false positives.

🧭 Rule of thumb

- If you want **high confidence, fewer but stronger terms**:
  → **FDR < 0.01**.

- If you want **a balance of discovery and reliability**:
  → **FDR < 0.05** (the most standard).
- If you're doing **exploratory analysis** and don't mind more false positives:
  → **FDR < 0.10**.

📝 Example in GOATOOLS

python Copy code

```python
for r in results:
    if r.p_fdr_bh < 0.05:  # strict, standard cutoff
        print(r.GO, r.name, r.p_fdr_bh, r.study_items)
```

✅ So, **FDR (Benjamini–Hochberg) < 0.05** is the most reasonable "strict" cutoff to report, with < 0.01 if you want to be very conservative.

Do you want me to also show you how to **sort results by enrichment strength** (e.g. by FDR or fold enrichment), so you see the strongest terms first instead of just filtering?

HOw should I modify the script to set
→ FDR < 0.01. ?

Nice and simple 👍 — you don't need to change the `GOEnrichmentStudy` setup at all. You just change the filtering step when you inspect the results.

Right now you have something like:

python Copy code

```python
results = goea.run_study(study_genes)

for r in results:
    if r.p_fdr_bh < 0.1:  # <-- you had a high permissive cutoff
        print(f"{r.GO} — {r.name} (p={r.p_fdr_bh:.3e})")
```

✅ Strict FDR < 0.01 cutoff

python Copy code

```python
results = goea.run_study(study_genes)

for r in results:
    if r.p_fdr_bh < 0.01:   # strict threshold
        print(f"{r.GO} — {r.name} (FDR={r.p_fdr_bh:.3e})")
```

🧭 Tip: Sort by significance

You might want to see the strongest terms first:

python Copy code

```python
sig_results = [r for r in results if r.p_fdr_bh < 0.01]
sig_results = sorted(sig_results, key=lambda x: x.p_fdr_bh)

for r in sig_results:
    print(f"{r.GO} — {r.name} (FDR={r.p_fdr_bh:.2e})")
```

That's it — just filter on `r.p_fdr_bh < 0.01`.

Would you like me to merge this filtering into the **GO → Gene → Direct Annotations** report we built earlier, so the output is already cleaned to FDR < 0.01?

Can we order enrichment results to group together those in the same branch e.g. here response to ionizing radition belongs with cellular response to ionising radiation, not with response to actinomycin D

GO:0071479 – cellular response to ionizing radiation (p=9.859e-05)
P38398 → ['GO:0071479 cellular response to ionizing radiation']
P04637 → ['GO:0071480 cellular response to gamma radiation', 'GO:0071479 cellular response to ionizing radiation']
Q00987 → ['GO:0071479 cellular response to ionizing radiation', 'GO:0071480 cellular response to gamma radiation']

GO:0072717 – cellular response to actinomycin D (p=9.859e-05)
Q00987 → ['GO:0072717 cellular response to actinomycin D']
P04637 → ['GO:0072717 cellular response to actinomycin D']

GO:0072716 – response to actinomycin D (p=1.315e-04)
Q00987 → ['GO:0072716 response to actinomycin D', 'GO:0072717 cellular response to actinomycin D']
P04637 → ['GO:0072717 cellular response to actinomycin D', 'GO:0072716 response to actinomycin D']

GO:0010212 – response to ionizing radiation (p=1.861e-04)
P38398 → ['GO:0010212 response to ionizing radiation', 'GO:0071479 cellular response to ionizing radiation']
P04637 → ['GO:0071480 cellular response to gamma radiation', 'GO:0010212 response to ionizing radiation', 'GO:0010332 response to gamma radiation', 'GO:0071479 cellular response to ionizing radiation', 'GO:0010165 response to X-ray']
Q00987 → ['GO:0010212 response to ionizing

radiation', 'GO:0010332 response to gamma radiation', 'GO:0071479 cellular response to ionizing radiation', 'GO:0071480 cellular response to gamma radiation']

Yes 👍 — what you're asking is to **cluster/group enriched GO terms by their ontology branch**, instead of just showing them in flat p-value order.

🔬 How to do it with GOATOOLS

Each term in `go_dag` knows its **parents** and **children**.
So you can:

1. Take your significant results (e.g. FDR < 0.01).
2. For each term, get its *ancestor chain* up to the root.
   - E.g. `go_dag[r.GO].get_all_parents()`
3. Use these ancestry paths to group terms that share a **common ancestor** (like `response to radiation`).
4. Within each branch, sort by p-value.

🛠️ Example Code

python Copy code

```python
from collections import defaultdict

# Step 1: select significant results
sig_results = [r for r in results if r.p_fdr_bh < 0.01]

# Step 2: group by a "branch root" — e.g. their topmost biological process
groups = defaultdict(list)

for r in sig_results:
    term = go_dag[r.GO]
    # get all ancestors (parents up to the root)
    ancestors = term.get_all_parents()
    # choose a "branch root": e.g. the most specific ancestor containing "
    branch_root = None
    for anc in ancestors:
        if "response" in go_dag[anc].name:
            branch_root = anc
            break
    if branch_root is None:
        branch_root = r.GO  # fallback to self
    groups[branch_root].append(r)

# Step 3: print grouped report
for root, terms in groups.items():
    print(f"\nBranch: {root} — {go_dag[root].name}")
    # sort terms by p-value
    for r in sorted(terms, key=lambda x: x.p_fdr_bh):
        print(f"  {r.GO} — {r.name}  (FDR={r.p_fdr_bh:.2e})")
        for g in r.study_items:
            direct_terms = [t for t in assoc.get(g, []) if t == r.GO or t
            names = [f"{tid} {go_dag[tid].name}" for tid in direct_terms]
            print(f"    {g} → {names}")
```

📊 What this gives you

Instead of your example being scattered:

- `cellular response to ionizing radiation`
- `response to actinomycin D`
- `response to ionizing radiation`

You'll get grouping like:

vbnet Copy code

```vbnet
Branch: GO:0010212 — response to ionizing radiation
  GO:0071479 — cellular response to ionizing radiation
  GO:0071480 — cellular response to gamma radiation
  GO:0010165 — response to X-ray
  GO:0010212 — response to ionizing radiation

Branch: GO:0072716 — response to actinomycin D
  GO:0072716 — response to actinomycin D
  GO:0072717 — cellular response to actinomycin D
```

⚡ This "branch root" selection is customizable. You could:

- Use the **most general ancestor (closest to root)** to form big clusters.
- Or pick the **lowest common ancestor (LCA)** of related terms.

👉 Do you want me to make a version that **auto-finds the lowest common ancestor (LCA)** for groups of enriched terms (so you don't have to hard-code "response" keywords)? That would give you much cleaner automatic clustering.

Can you give me full code for running branch grouping. Separately I would like code for LCA

Perfect — let's build both versions. I'll keep them self-contained and show you where to plug in your study gene list.

We assume you already have:

- `go_dag = GODag("go-basic.obo")`
- `assoc = read_gaf(open("goa_human.gaf"))` (direct annotations)
- `population = list(assoc.keys())`
- `study_genes = [...]` (your UniProt IDs)
- `results = goea.run_study(study_genes)`

1️⃣ Branch Grouping (by ancestor keyword or chosen ancestor)

This approach groups enriched terms under a "branch root" ancestor (e.g. the top "response to X" term).

Copy code

python

```python
from collections import defaultdict

def group_by_branch(sig_results, go_dag, assoc, keyword="response"):
    """Group enriched terms by a branch ancestor containing `keyword` in i
    groups = defaultdict(list)

    for r in sig_results:
        term = go_dag[r.GO]
        ancestors = term.get_all_parents()
        branch_root = None
        # pick first ancestor whose name contains the keyword
        for anc in ancestors:
            if keyword in go_dag[anc].name.lower():
                branch_root = anc
                break
        if branch_root is None:
            branch_root = r.GO
        groups[branch_root].append(r)

    # Pretty print
    for root, terms in groups.items():
        print(f"\nBranch: {root} — {go_dag[root].name}")
        for r in sorted(terms, key=lambda x: x.p_fdr_bh):
            print(f"  {r.GO} — {r.name}  (FDR={r.p_fdr_bh:.2e})")
            for g in r.study_items:
                direct_terms = [t for t in assoc.get(g, [])
                                if t == r.GO or t in go_dag[r.GO].get_all_
                names = [f"{tid} {go_dag[tid].name}" for tid in direct_ter
                print(f"    {g} → {names}")

# Example usage:
sig_results = [r for r in results if r.p_fdr_bh < 0.01]
group_by_branch(sig_results, go_dag, assoc, keyword="response")
```

2️⃣ Grouping by Lowest Common Ancestor (LCA)

This is stricter: we cluster enriched terms by the lowest common ancestor of pairs of enriched terms.

python Copy code

```python
from collections import defaultdict

def lowest_common_ancestor(go_ids, go_dag):
    """Return the lowest common ancestor GO term for a list of GO IDs."""
    if not go_ids:
        return None
    # Start from ancestors of the first term
    common_ancs = set(go_dag[go_ids[0]].get_all_parents()) | {go_ids[0]}
    for go in go_ids[1:]:
        ancs = set(go_dag[go].get_all_parents()) | {go}
        common_ancs &= ancs
    if not common_ancs:
        return None
    # Pick the ancestor with maximum depth (lowest in DAG)
    return max(common_ancs, key=lambda go: go_dag[go].depth)

def group_by_lca(sig_results, go_dag, assoc):
```

```
    """Group enriched terms by their lowest common ancestor."""
    groups = defaultdict(list)

    # naive: compute LCA for each term individually (can be optimized)
    for r in sig_results:
        lca = lowest_common_ancestor([r.GO], go_dag)
        groups[lca].append(r)

    # print report
    for root, terms in groups.items():
        rootname = go_dag[root].name if root else "None"
        print(f"\nGroup (LCA): {root} — {rootname}")
        for r in sorted(terms, key=lambda x: x.p_fdr_bh):
            print(f"  {r.GO} — {r.name}  (FDR={r.p_fdr_bh:.2e})")
            for g in r.study_items:
                direct_terms = [t for t in assoc.get(g, [])
                                if t == r.GO or t in go_dag[r.GO].get_all_
                names = [f"{tid} {go_dag[tid].name}" for tid in direct_ter
                print(f"    {g} → {names}")

# Example usage
sig_results = [r for r in results if r.p_fdr_bh < 0.01]
group_by_lca(sig_results, go_dag, assoc)
```

🔑 Key Differences

- **Branch grouping** → you control the ancestor (e.g. any "response" term). Good if you want to pull together semantically related terms.
- **LCA grouping** → purely structural: cluster terms by their lowest shared ancestor in the DAG.

⚡ Do you want me to extend the **LCA version** to group *all enriched terms together* under shared LCAs (like "response to radiation" grouping multiple children), instead of treating each term individually? That would give you exactly the "radiation branch" vs. "actinomycin branch" layout automatically.

yes please

Perfect ✅ — let's build the **full LCA-based clustering**, so enriched terms that share a "lowest common ancestor" (LCA) get grouped together.

That way, things like *"response to ionizing radiation"*, *"cellular response to ionizing radiation"*, *"response to gamma radiation"*, etc. will naturally cluster into one branch, separate from *"response to actinomycin D"*.

🔧 Full Code for LCA Grouping

python Copy code

```python
from collections import defaultdict
```

```python
def lowest_common_ancestor(go_ids, go_dag):
    """Return the lowest common ancestor GO term for a list of GO IDs."""
    if not go_ids:
        return None
    # start with all ancestors (including self) of the first term
    common_ancs = set(go_dag[go_ids[0]].get_all_parents()) | {go_ids[0]}
    for go in go_ids[1:]:
        ancs = set(go_dag[go].get_all_parents()) | {go}
        common_ancs &= ancs
    if not common_ancs:
        return None
    # pick the ancestor deepest in the DAG
    return max(common_ancs, key=lambda go: go_dag[go].depth)


def group_by_lca(sig_results, go_dag, assoc, fdr_cutoff=0.01):
    """Group enriched terms into clusters using their lowest common ancest
    # filter
    sig_results = [r for r in sig_results if r.p_fdr_bh < fdr_cutoff]

    # Build groups: for each pair of terms, find their LCA
    clusters = defaultdict(list)
    seen = set()

    for r in sig_results:
        if r.GO in seen:
            continue
        # Try to find other terms that share an LCA with r
        group_terms = [r]
        for s in sig_results:
            if s is r or s.GO in seen:
                continue
            lca = lowest_common_ancestor([r.GO, s.GO], go_dag)
            if lca:
                group_terms.append(s)
        # assign cluster by LCA of the group
        go_ids = [x.GO for x in group_terms]
        lca = lowest_common_ancestor(go_ids, go_dag)
        if lca is None:
            lca = r.GO
        clusters[lca].extend(group_terms)
        seen.update(go_ids)

    # Pretty print clusters
    for root, terms in clusters.items():
        print(f"\nCluster root (LCA): {root} — {go_dag[root].name}")
        for r in sorted(terms, key=lambda x: x.p_fdr_bh):
            print(f"  {r.GO} — {r.name}  (FDR={r.p_fdr_bh:.2e})")
            for g in r.study_items:
                # show direct annotations explaining contribution
                direct_terms = [t for t in assoc.get(g, [])
                                if t == r.GO or t in go_dag[r.GO].get_all_
                names = [f"{tid} {go_dag[tid].name}" for tid in direct_ter
                print(f"    {g} → {names}")
```

🔎 Usage Example

python Copy code

```
# Run enrichment first
results = goea.run_study(study_genes)

# Group significant results
group_by_lca(results, go_dag, assoc, fdr_cutoff=0.01)
```

📊 Example Output (mocked)

pgsql Copy code

```pgsql
Cluster root (LCA): GO:0010212 – response to ionizing radiation
  GO:0071479 – cellular response to ionizing radiation  (FDR=9.86e-05)
    P04637 → [GO:0071479 cellular response to ionizing radiation]
  GO:0071480 – cellular response to gamma radiation  (FDR=1.05e-04)
    P04637 → [GO:0071480 cellular response to gamma radiation]
  GO:0010165 – response to X-ray  (FDR=2.00e-04)
    P04637 → [GO:0010165 response to X-ray]

Cluster root (LCA): GO:0072716 – response to actinomycin D
  GO:0072716 – response to actinomycin D  (FDR=1.31e-04)
    Q00987 → [GO:0072716 response to actinomycin D]
  GO:0072717 – cellular response to actinomycin D  (FDR=9.86e-05)
    P04637 → [GO:0072717 cellular response to actinomycin D]
```

👉 Would you like me to also make this export to a **hierarchical tree (JSON or CSV)** instead of just printing, so you could visualize clusters (e.g. in Cytoscape or a Sankey diagram)?

Instead of clustering I'm seeing everything under Cluster root (LCA): GO:0008150 – biological_process

Ah, yes 👍 — that's exactly what happens if the **lowest common ancestor (LCA)** across enriched terms is too general:

- `GO:0008150 – biological_process` is the **root** of almost everything in the BP namespace.
- If you compute LCA naïvely across many terms, you'll almost always end up at the root.

🔬 Why this happens

My earlier code was grouping too aggressively:

- It tried to find an LCA across *all terms in the group*.
- Since all "response to X" terms eventually trace back to **biological_process**, that became the LCA.

✅ Better approach: cluster by pairwise LCAs and exclude root

Instead of computing one LCA for all, we should:

1. Take pairs of enriched terms.
2. Find their LCA.
3. If the LCA is very general (like `biological_process`, `molecular_function`, `cellular_component`) → ignore it.
4. Group under the **lowest non-root ancestor** that still unites them.

🔧 Revised Code

python Copy code

```python
from collections import defaultdict

# Define root terms to ignore
ROOT_TERMS = {"GO:0008150", "GO:0003674", "GO:0005575"}  # BP, MF, CC

def lowest_common_ancestor(go_ids, go_dag):
    """Return the lowest common ancestor (non-root) of a list of GO terms.
    if not go_ids:
        return None
    common_ancs = set(go_dag[go_ids[0]].get_all_parents()) | {go_ids[0]}
    for go in go_ids[1:]:
        ancs = set(go_dag[go].get_all_parents()) | {go}
        common_ancs &= ancs
    if not common_ancs:
        return None
    # pick deepest ancestor not in ROOT_TERMS
    valid_ancs = [go for go in common_ancs if go not in ROOT_TERMS]
    if not valid_ancs:
        return None
    return max(valid_ancs, key=lambda go: go_dag[go].depth)


def group_by_lca(sig_results, go_dag, assoc, fdr_cutoff=0.01):
    sig_results = [r for r in sig_results if r.p_fdr_bh < fdr_cutoff]

    clusters = defaultdict(list)
    seen = set()

    for r in sig_results:
        if r.GO in seen:
            continue
        # build a cluster of terms that share an LCA with r
        group_terms = [r]
        for s in sig_results:
            if s is r or s.GO in seen:
                continue
            lca = lowest_common_ancestor([r.GO, s.GO], go_dag)
            if lca:
                group_terms.append(s)
        # assign cluster root as the LCA of this group
        go_ids = [x.GO for x in group_terms]
        lca = lowest_common_ancestor(go_ids, go_dag)
        if lca is None:
            lca = r.GO
        clusters[lca].extend(group_terms)
        seen.update(go_ids)
```

```
    # print grouped clusters
    for root, terms in clusters.items():
        rootname = go_dag[root].name if root else "None"
        print(f"\nCluster root (LCA): {root} — {rootname}")
        for r in sorted(terms, key=lambda x: x.p_fdr_bh):
            print(f"  {r.GO} — {r.name}  (FDR={r.p_fdr_bh:.2e})")
            for g in r.study_items:
                direct_terms = [t for t in assoc.get(g, [])
                                if t == r.GO or t in go_dag[r.GO].get_all_
                names = [f"{tid} {go_dag[tid].name}" for tid in direct_ter
                print(f"    {g} → {names}")
```

🔎 Expected Output

Now, instead of everything under **biological_process**, you should see more meaningful LCAs like:

pgsql Copy code

```pgsql
Cluster root (LCA): GO:0010212 — response to ionizing radiation
  GO:0071479 — cellular response to ionizing radiation
  GO:0071480 — cellular response to gamma radiation
  GO:0010165 — response to X-ray

Cluster root (LCA): GO:0072716 — response to actinomycin D
  GO:0072716 — response to actinomycin D
  GO:0072717 — cellular response to actinomycin D
```

👉 Do you want me to add an option where the **cluster root name is automatically promoted to the most specific ancestor containing "response"** (so all "response to radiation" terms explicitly end up under that branch)?

Hmm - that doesn't sound good. Still too general. I want clustering under terms already on the enrichment list - with high confidence (perhaps tuning clustering on confidence would be ideal?). Aim is to find some optimal informative clustering that is not too absract.

Got it 👍 — what you want is not to float all the way up the DAG (which gets you *biological process*), but to **cluster enriched terms under other enriched terms that are already in your significant list**.

That way, the "cluster roots" are **real enriched terms**, not abstract ancestors.
This is exactly what tools like **REVIGO** or **EnrichmentMap in Cytoscape** try to do — keep the grouping informative and avoid collapsing to useless high-level categories.

🔬 Strategy for *Enriched-Term–Based Clustering*

1. Take your enriched terms (FDR < cutoff).

2. For each term, check if it is an **ancestor of another enriched term** (within your sig list).
   - If yes, treat it as a **cluster root**.
   - The children get grouped under it.
3. Keep only terms that are actually enriched, so cluster roots are interpretable.
4. Optionally tune clustering by *confidence*:
   - Require cluster root to be very significant (e.g. FDR < 0.001).
   - Allow looser members (FDR < 0.05) to be grouped under it.

🔧 Code Example

python Copy code

```python
from collections import defaultdict

def cluster_by_enriched_ancestors(results, go_dag, assoc,
                                  root_cutoff=0.001, member_cutoff=0.05):
    """Cluster enriched GO terms under other enriched terms that are their
    sig_terms = [r for r in results if r.p_fdr_bh < member_cutoff]
    sig_map = {r.GO: r for r in sig_terms}

    clusters = defaultdict(list)

    for r in sig_terms:
        # look for enriched ancestors of this term
        ancestors = go_dag[r.GO].get_all_parents()
        enriched_ancestors = [a for a in ancestors if a in sig_map]

        # pick the most significant enriched ancestor as cluster root
        root = None
        if enriched_ancestors:
            root = min(enriched_ancestors, key=lambda go: sig_map[go].p_fd
            if sig_map[root].p_fdr_bh > root_cutoff:
                root = None
        if root is None:
            root = r.GO

        clusters[root].append(r)

    # print clusters
    for root, terms in clusters.items():
        rootr = sig_map[root]
        print(f"\nCluster root: {root} — {rootr.name} (FDR={rootr.p_fdr_bh
        for r in sorted(terms, key=lambda x: x.p_fdr_bh):
            print(f"  {r.GO} — {r.name}  (FDR={r.p_fdr_bh:.2e})")
            for g in r.study_items:
                direct_terms = [t for t in assoc.get(g, [])
                                if t == r.GO or t in go_dag[r.GO].get_all_
                names = [f"{tid} {go_dag[tid].name}" for tid in direct_ter
                print(f"    {g} → {names}")
```

🔎 Example Behavior

If your enrichment contains:

- **GO:0010212 – response to ionizing radiation (FDR=1e-04)**
- **GO:0071479 – cellular response to ionizing radiation (FDR=9.8e-05)**

- **GO:0071480 – cellular response to gamma radiation (FDR=1.0e-04)**
- **GO:0072716 – response to actinomycin D (FDR=1.3e-04)**
- **GO:0072717 – cellular response to actinomycin D (FDR=9.8e-05)**

Then clusters would look like:

pgsql Copy code

```
Cluster root: GO:0010212 – response to ionizing radiation (FDR=1.00e-04)
  GO:0071479 – cellular response to ionizing radiation  (FDR=9.86e-05)
  GO:0071480 – cellular response to gamma radiation  (FDR=1.00e-04)

Cluster root: GO:0072716 – response to actinomycin D (FDR=1.31e-04)
  GO:0072717 – cellular response to actinomycin D (FDR=9.86e-05)
```

🧭 Why this is better

- Cluster roots are **enriched terms themselves** (not abstract ancestors like *biological process*).
- You can tune strictness:
  - `root_cutoff` controls how significant a term must be to serve as a root.
  - `member_cutoff` controls how loose terms can be and still get grouped.

👉 Would you like me to extend this to **output a JSON tree** (cluster root → children → genes) so you can visualize it in Cytoscape or another graph tool, instead of just printing?

Lets print for now, but a JSON tree option would be cools to have

Perfect 🙌 — let's do both:

1. **Readable console output** (for inspection now).
2. **Optional JSON tree export** you can later feed into Cytoscape, D3.js, or other visualization tools.

🔧 Updated Enriched-Term Clustering Code

python Copy code

```python
from collections import defaultdict
import json

def cluster_by_enriched_ancestors(results, go_dag, assoc,
                                  root_cutoff=0.001, member_cutoff=0.05,
                                  export_json=None):
    """
    Cluster enriched GO terms under other enriched terms that are their an

    Parameters
    ----------
```

```
    results : list
        GOATOOLS enrichment results
    go_dag : GODag
        Loaded GO DAG
    assoc : dict
        Gene → direct GO annotations
    root_cutoff : float
        Max FDR for a term to qualify as a cluster root
    member_cutoff : float
        Max FDR for a term to be included as a cluster member
    export_json : str or None
        If filename, export clusters to JSON file
    """
    # Filter enriched terms
    sig_terms = [r for r in results if r.p_fdr_bh < member_cutoff]
    sig_map = {r.GO: r for r in sig_terms}

    clusters = defaultdict(list)

    # Build clusters
    for r in sig_terms:
        ancestors = go_dag[r.GO].get_all_parents()
        enriched_ancestors = [a for a in ancestors if a in sig_map]

        root = None
        if enriched_ancestors:
            # pick most significant enriched ancestor
            root = min(enriched_ancestors, key=lambda go: sig_map[go].p_fd
            if sig_map[root].p_fdr_bh > root_cutoff:
                root = None
        if root is None:
            root = r.GO

        clusters[root].append(r)

    # ---- PRINT ----
    for root, terms in clusters.items():
        rootr = sig_map[root]
        print(f"\nCluster root: {root} — {rootr.name} (FDR={rootr.p_fdr_bh
        for r in sorted(terms, key=lambda x: x.p_fdr_bh):
            print(f"  {r.GO} — {r.name}  (FDR={r.p_fdr_bh:.2e})")
            for g in r.study_items:
                direct_terms = [t for t in assoc.get(g, [])
                                if t == r.GO or t in go_dag[r.GO].get_all_
                names = [f"{tid} {go_dag[tid].name}" for tid in direct_ter
                print(f"    {g} → {names}")

    # ---- JSON EXPORT ----
    if export_json:
        json_clusters = {}
        for root, terms in clusters.items():
            rootr = sig_map[root]
            json_clusters[root] = {
                "name": rootr.name,
                "FDR": rootr.p_fdr_bh,
                "members": []
            }
            for r in sorted(terms, key=lambda x: x.p_fdr_bh):
                json_clusters[root]["members"].append({
                    "id": r.GO,
                    "name": r.name,
```

```
                    "FDR": r.p_fdr_bh,
                    "genes": {
                        g: [f"{tid} {go_dag[tid].name}" for tid in assoc.g
                            if tid == r.GO or tid in go_dag[r.GO].get_all_
                        for g in r.study_items
                    }
                })
        with open(export_json, "w") as f:
            json.dump(json_clusters, f, indent=2)
        print(f"\nExported clusters to {export_json}")
```

🔎 Example Usage

python Copy code

```python
results = goea.run_study(study_genes)

# Print to screen
cluster_by_enriched_ancestors(results, go_dag, assoc,
                              root_cutoff=0.001, member_cutoff=0.05)

# Also export JSON for Cytoscape/D3/etc.
cluster_by_enriched_ancestors(results, go_dag, assoc,
                              root_cutoff=0.001, member_cutoff=0.05,
                              export_json="clusters.json")
```

📊 Example Output (console)

pgsql Copy code

```
Cluster root: GO:0010212 — response to ionizing radiation (FDR=1.00e-04)
  GO:0071479 — cellular response to ionizing radiation  (FDR=9.86e-05)
    P04637 → [GO:0071479 cellular response to ionizing radiation]
  GO:0071480 — cellular response to gamma radiation  (FDR=1.00e-04)
    P04637 → [GO:0071480 cellular response to gamma radiation]

Cluster root: GO:0072716 — response to actinomycin D (FDR=1.31e-04)
  GO:0072717 — cellular response to actinomycin D (FDR=9.86e-05)
    Q00987 → [GO:0072717 cellular response to actinomycin D]
```

📂 Example JSON Output

json Copy code

```json
{
  "GO:0010212": {
    "name": "response to ionizing radiation",
    "FDR": 0.0001,
    "members": [
      {
        "id": "GO:0071479",
        "name": "cellular response to ionizing radiation",
        "FDR": 0.0000986,
        "genes": {
          "P04637": ["GO:0071479 cellular response to ionizing radiation"]
        }
```

```
      }
    ]
  }
}
```

👉 Do you want me to also include **fold enrichment** (study ratio / population ratio) in both the printout and JSON, so you can rank terms not only by FDR but also by effect size?

Limit clusters to top 10 enrichment results, but include anything significant under these.

Got it 👍 — you want the **top N most significant enriched terms** to serve as *cluster roots* (say top 10 by FDR), and then include **any other significant enriched terms that descend from them**.

That way, you get a focused set of "main topics" but still retain detailed child terms underneath.

🔧 Updated Code: Top N Root Clusters

python Copy code

```python
from collections import defaultdict
import json

def cluster_topN_roots(results, go_dag, assoc,
                       top_n=10, fdr_cutoff=0.05,
                       export_json=None):
    """
    Cluster enrichment results under the top-N most significant terms.

    Parameters
    ----------
    results : list
        GOATOOLS enrichment results
    go_dag : GODag
    assoc : dict
        Gene → direct GO annotations
    top_n : int
        Number of most significant terms to use as cluster roots
    fdr_cutoff : float
        Max FDR for a term to be considered significant (included under a
    export_json : str or None
        If filename, export clusters to JSON
    """
    # Significant terms only
    sig_results = [r for r in results if r.p_fdr_bh < fdr_cutoff]
    sig_map = {r.GO: r for r in sig_results}

    # Pick top N roots by FDR
    roots = sorted(sig_results, key=lambda r: r.p_fdr_bh)[:top_n]
```

```
root_ids = {r.GO for r in roots}

clusters = defaultdict(list)

for r in sig_results:
    # assign term to first root ancestor it has
    assigned = False
    for root in roots:
        if root.GO == r.GO or root.GO in go_dag[r.GO].get_all_parents(
            clusters[root.GO].append(r)
            assigned = True
            break
    if not assigned:
        # no root found → leave as its own cluster
        clusters[r.GO].append(r)

# ---- PRINT ----
for root_id, terms in clusters.items():
    rootr = sig_map.get(root_id, None)
    if rootr:
        print(f"\nCluster root: {root_id} — {rootr.name} (FDR={rootr.p
    else:
        print(f"\nCluster root: {root_id} — {go_dag[root_id].name}")
    for r in sorted(terms, key=lambda x: x.p_fdr_bh):
        print(f"  {r.GO} — {r.name}  (FDR={r.p_fdr_bh:.2e})")
        for g in r.study_items:
            direct_terms = [t for t in assoc.get(g, [])
                            if t == r.GO or t in go_dag[r.GO].get_all_
            names = [f"{tid} {go_dag[tid].name}" for tid in direct_ter
            print(f"    {g} → {names}")

# ---- JSON EXPORT ----
if export_json:
    json_clusters = {}
    for root_id, terms in clusters.items():
        rootr = sig_map.get(root_id, None)
        json_clusters[root_id] = {
            "name": rootr.name if rootr else go_dag[root_id].name,
            "FDR": rootr.p_fdr_bh if rootr else None,
            "members": []
        }
        for r in sorted(terms, key=lambda x: x.p_fdr_bh):
            json_clusters[root_id]["members"].append({
                "id": r.GO,
                "name": r.name,
                "FDR": r.p_fdr_bh,
                "genes": {
                    g: [f"{tid} {go_dag[tid].name}" for tid in assoc.g
                        if tid == r.GO or tid in go_dag[r.GO].get_all_
                    for g in r.study_items
                }
            })
    with open(export_json, "w") as f:
        json.dump(json_clusters, f, indent=2)
    print(f"\nExported clusters to {export_json}")
```

🔎 Usage Example

python Copy code

```
results = goea.run_study(study_genes)

# Cluster only under top 10 most significant roots
cluster_topN_roots(results, go_dag, assoc,
                   top_n=10, fdr_cutoff=0.05,
                   export_json="clusters_top10.json")
```

📊 Expected Behavior

- Take top 10 GO terms (lowest FDR).
- Each becomes a **cluster root**.
- Any other significant enriched terms that descend from these roots are grouped under them.
- Roots remain interpretable because they are actual enriched terms.

Example:

pgsql Copy code

```
Cluster root: GO:0010212 — response to ionizing radiation (FDR=1.00e-04)
  GO:0071479 — cellular response to ionizing radiation (FDR=9.86e-05)
  GO:0071480 — cellular response to gamma radiation (FDR=1.00e-04)
  GO:0010165 — response to X-ray (FDR=2.00e-04)

Cluster root: GO:0072716 — response to actinomycin D (FDR=1.31e-04)
  GO:0072717 — cellular response to actinomycin D (FDR=9.86e-05)
```

👉 Do you want me to also add a **fold enrichment score** to each term (both print and JSON), so you can rank not only by FDR but also by effect size within clusters?

sure

Awesome — let's extend the clustering code so it also reports **fold enrichment** for each GO term.

🔬 Fold Enrichment in GOATOOLS

Each enrichment result object (`r`) has:

- `r.study_count` → number of study genes annotated to this term
- `r.study_n` → total number of study genes
- `r.pop_count` → number of population genes annotated to this term
- `r.pop_n` → total number of population genes

Fold enrichment =

$$\mathrm{FE} = \frac{(study_count / study_n)}{(pop_count / pop_n)}$$

🔧 Updated Code with Fold Enrichment

python Copy code

```python
from collections import defaultdict
import json

def calc_fold_enrichment(r):
    """Calculate fold enrichment for a GOATOOLS result record."""
    if r.pop_count == 0 or r.study_n == 0 or r.pop_n == 0:
        return None
    study_frac = r.study_count / r.study_n
    pop_frac = r.pop_count / r.pop_n
    return study_frac / pop_frac if pop_frac > 0 else None


def cluster_topN_roots(results, go_dag, assoc,
                       top_n=10, fdr_cutoff=0.05,
                       export_json=None):
    """Cluster enrichment results under the top-N most significant terms, 

    # Significant terms only
    sig_results = [r for r in results if r.p_fdr_bh < fdr_cutoff]
    sig_map = {r.GO: r for r in sig_results}

    # Pick top N roots by FDR
    roots = sorted(sig_results, key=lambda r: r.p_fdr_bh)[:top_n]

    clusters = defaultdict(list)

    for r in sig_results:
        # assign to first root ancestor it has
        assigned = False
        for root in roots:
            if root.GO == r.GO or root.GO in go_dag[r.GO].get_all_parents(
                clusters[root.GO].append(r)
                assigned = True
                break
        if not assigned:
            clusters[r.GO].append(r)

    # ---- PRINT ----
    for root_id, terms in clusters.items():
        rootr = sig_map.get(root_id)
        fe_root = calc_fold_enrichment(rootr) if rootr else None
        print(f"\nCluster root: {root_id} — {rootr.name} "
              f"(FDR={rootr.p_fdr_bh:.2e}, FE={fe_root:.2f})")
        for r in sorted(terms, key=lambda x: x.p_fdr_bh):
            fe = calc_fold_enrichment(r)
            print(f"  {r.GO} — {r.name}  (FDR={r.p_fdr_bh:.2e}, FE={fe:.2f
            for g in r.study_items:
                direct_terms = [t for t in assoc.get(g, [])
                                if t == r.GO or t in go_dag[r.GO].get_all_
                names = [f"{tid} {go_dag[tid].name}" for tid in direct_ter
                print(f"    {g} → {names}")

    # ---- JSON EXPORT ----
    if export_json:
        json_clusters = {}
        for root_id, terms in clusters.items():
            rootr = sig_map.get(root_id, None)
            json_clusters[root_id] = {
                "name": rootr.name if rootr else go_dag[root_id].name,
```

```
                "FDR": rootr.p_fdr_bh if rootr else None,
                "FE": calc_fold_enrichment(rootr) if rootr else None,
                "members": []
            }
            for r in sorted(terms, key=lambda x: x.p_fdr_bh):
                json_clusters[root_id]["members"].append({
                    "id": r.GO,
                    "name": r.name,
                    "FDR": r.p_fdr_bh,
                    "FE": calc_fold_enrichment(r),
                    "genes": {
                        g: [f"{tid} {go_dag[tid].name}" for tid in assoc.g
                            if tid == r.GO or tid in go_dag[r.GO].get_all_
                        for g in r.study_items
                    }
                })
        with open(export_json, "w") as f:
            json.dump(json_clusters, f, indent=2)
        print(f"\nExported clusters to {export_json}")
```

🔎 Usage Example

python Copy code

```python
results = goea.run_study(study_genes)

# Cluster top 10 roots with fold enrichment
cluster_topN_roots(results, go_dag, assoc,
                   top_n=10, fdr_cutoff=0.05,
                   export_json="clusters_top10.json")
```

📊 Example Output

pgsql Copy code

```pgsql
Cluster root: GO:0010212 — response to ionizing radiation (FDR=1.00e-04, F
  GO:0071479 — cellular response to ionizing radiation  (FDR=9.86e-05, FE=
    P04637 → [GO:0071479 cellular response to ionizing radiation]
  GO:0071480 — cellular response to gamma radiation  (FDR=1.00e-04, FE=7.9
    P04637 → [GO:0071480 cellular response to gamma radiation]

Cluster root: GO:0072716 — response to actinomycin D (FDR=1.31e-04, FE=6.4
  GO:0072717 — cellular response to actinomycin D (FDR=9.86e-05, FE=7.02)
    Q00987 → [GO:0072717 cellular response to actinomycin D]
```

👉 Would you also like me to **sort clusters by fold enrichment of the root** (instead of FDR), so you can highlight the "strongest biological signals" first?

Doesn't GOATOOLS already support drill down and graphs? Are we reimplementing?

Excellent question 👏 — and you're absolutely right to ask it.

You're not missing anything — **GOATOOLS *does* have built-in drill-down and graph features**.
What we've been building here is a **custom extension** on top of GOATOOLS' core functionality, to give you:

- hierarchical **clustering across enriched terms**, not just parent–child structure;
- and detailed per-gene → per-direct-annotation reporting that's not printed by default.

Let's unpack this clearly:

🔬 What GOATOOLS already does

GOATOOLS provides quite a bit "out of the box":

1. Drill-down (hierarchical GO DAG traversal)

Every `GODag` term (`go_dag["GO:XXXXXXX"]`) exposes:

python Copy code

```python
term.get_all_children()   # descendants (more specific)
term.get_all_parents()    # ancestors (more general)
```

and you can use these to navigate or filter terms.

When you run enrichment with:

python Copy code

```python
goea = GOEnrichmentStudy(population, assoc, go_dag, propagate_counts=True)
```

GOATOOLS automatically propagates counts upward — so enrichment implicitly includes indirect annotations.

---

2. Built-in visualizations

GOATOOLS can render **GO graphs** (using Graphviz) for individual enriched terms or full results:

python Copy code

```python
goea.wr_xlsx("enrichment.xlsx")  # Excel with hierarchy info
goea.wr_txt("enrichment.txt", prt_if=True)  # text summary
goea.wr_graph("go_enrichment.png", results, min_ratio=0.05)
```

The `.wr_graph()` method gives you a **GO DAG visualization**:

- Nodes = GO terms (colored by significance)
- Edges = parent/child links
- You can limit the graph to just significant terms

These are "drill-down" plots — you can visually trace enriched terms and their descendants.

---

3. Automatic drill-down table output

`wr_txt` or `wr_xlsx` reports already contain per-term statistics:

- `GO`, `name`, `p_uncorrected`, `p_fdr_bh`
- `enrichment`, `study_count`, `pop_count`
- `study_items` (your genes in that term)

So, for a standard analysis, you can already get:

enriched GO term → contributing genes

by just inspecting the text or Excel outputs.

🧠 What we added

Where our custom code goes beyond the built-in drill-down:

Feature	GOATOOLS native?	Our addition
GO → contributing genes	✅ (`study_items`)	same
Gene → direct vs indirect annotations	❌	✅ we used raw GAF to separate direct from propagated
Clustering enriched terms by structure	partial (DAG exists)	✅ we group by enriched ancestors or top-N roots
Custom branch grouping (e.g. "radiation", "drug response")	❌	✅ heuristic + LCA + top-N root logic
JSON tree export	❌	✅ for Cytoscape/D3 downstream use
Fold enrichment in cluster printouts	✅ (computed internally)	✅ we expose it in print/export

---

💡 Recommended workflow

If you want the **cleanest integration with GOATOOLS features**, you can:

1. Use GOATOOLS to generate a **graph and Excel summary**:

   python Copy code

   ```python
   goea.wr_graph("sig_terms.png", results, pval=0.01)
   goea.wr_xlsx("sig_terms.xlsx", results)
   ```

2. Use our **clustering + JSON** layer on top of that, for:
   - organizing significant results into meaningful branches
   - extracting per-gene annotation context

They complement each other — not duplicate work.
GOATOOLS' own visual graph is *term-centric*, whereas our code gives you a **cluster-centric drill-down**.

✅ In short:

- You're not reinventing the wheel — we're adding a steering wheel, dashboard, and GPS to GOATOOLS' engine 🚗
- GOATOOLS gives structure, p-values, and DAG traversal

- Our layer adds interpretability, clustering, and exportable summaries

---

Would you like me to show how to **combine GOATOOLS' built-in** `wr_graph` **visual output** with our clustering results — e.g. automatically export one `.png` GO DAG per cluster root (colored by FDR)?

I have a novel proposed extension to GO. A concept with associated genes mined from the literature, in GAF format. How can I fold this in to the above test?

Fantastic — now we're in very interesting territory 🔬

You have a **proposed new GO concept** (a term not yet in the official ontology), and you've already built a **mini annotation set** (gene → GO term associations) in **GAF format**.
You want to **test and analyze this concept in context** of the standard GO — e.g. see whether it behaves like an existing term, where it fits, and how it enriches.

Let's walk through how to "fold it in" cleanly.

🧩 Step 1. Add your concept to the GO DAG

You can dynamically extend the GO DAG in GOATOOLS by creating a new term and linking it to existing ones.

python

```python
from goatools.obo_parser import GODag, GOTerm

# load the normal GO DAG
go_dag = GODag("go-basic.obo")

# create a new term
new_go_id = "GO:NEW0001"
new_term = GOTerm()
new_term.id = new_go_id
new_term.name = "response to radiotherapy"  # your concept
new_term.namespace = "biological_process"

# link it to an existing parent term (e.g. "response to radiation")
parent_id = "GO:0010212"
new_term.parents = {go_dag[parent_id]}
new_term.level = go_dag[parent_id].level + 1
new_term.depth = go_dag[parent_id].depth + 1

# register it into the DAG
go_dag[new_go_id] = new_term
go_dag[parent_id].children.add(new_term)

print(f"Added {new_go_id} under {parent_id}")
```

Now your `go_dag` behaves as if that new concept is a legitimate GO term — fully navigable with parents and children.

📄 Step 2. Add your gene associations (GAF)

If your mined annotations are in GAF format (e.g. `my_concept.gaf`):

ruby Copy code

```ruby
UniProtKB   P04637  TP53        GO:NEW0001  PMID:XXXXXXX    EXP     Homo s
UniProtKB   P38398  BRCA1       GO:NEW0001  PMID:XXXXXXX    EXP     Homo s
```

You can merge them with your main annotation set:

python Copy code

```python
from goatools.associations import read_gaf

# existing annotations (e.g. goa_human.gaf)
assoc = read_gaf("goa_human.gaf.gz")

# read your extension GAF
ext_assoc = read_gaf("my_concept.gaf")

# merge
for gene, terms in ext_assoc.items():
    assoc.setdefault(gene, set()).update(terms)

print(f"Added {len(ext_assoc)} new gene annotations from extension.")
```

Now your association dict knows about your novel term.

🔬 Step 3. Run enrichment (with the extended GO)

python Copy code

```python
from goatools.goea.go_enrichment_ns import GOEnrichmentStudy

goea = GOEnrichmentStudy(
    population=list(assoc.keys()),
    assoc=assoc,
    obo_dag=go_dag,
    propagate_counts=True,
    alpha=0.05,
    methods=['fdr_bh']
)

results = goea.run_study(study_genes)
```

Your new term (`GO:NEW0001`) will now appear **just like any other GO term** in enrichment results — including FDR, fold enrichment, and per-gene drill-down.

🧠 Step 4. Analyze and visualize

You can: Copy code

- Check where it fits conceptually: `go_dag[new_go_id].get_all_parents()`

- See which standard GO terms are enriched together with it (potential redundancy or overlap).
- Use the same **clustering code** we built — it will treat your new term as part of the ontology and include it in clusters.

Example:

pgsql Copy code

```
Cluster root: GO:0010212 — response to ionizing radiation (FDR=1.00e-04)
  GO:NEW0001 — response to radiotherapy (FDR=3.20e-04, FE=5.62)
    P04637 → [GO:NEW0001 response to radiotherapy]
    P38398 → [GO:NEW0001 response to radiotherapy]
```

🧬 Step 5. Optional: test placement

If you're exploring **where to attach the new concept**, you can:

- Try linking it under several possible parents.
- Compare which parent leads to strongest co-enrichment or shared gene overlap.
- Use `go_dag[new_go_id].get_all_parents()` to see where it "makes sense" semantically.

This can even become a **data-driven proposal** for ontology placement — your enrichment behavior provides evidence for which branch it belongs to.

✅ Summary

Goal	Method
Add new concept	Create a `GOTerm`, attach to a parent, insert into DAG
Add annotations	Read your GAF → merge into `assoc` dict
Test enrichment	Run `GOEnrichmentStudy` as usual
Interpret results	Use clustering, drill-down, or GOATOOLS graphs
Optional export	Include new term in `.wr_graph()` or JSON clusters

Would you like me to show how to **validate that your new term behaves sensibly** — e.g. print its enrichment neighbors (terms sharing most genes) or show how often its genes overlap with existing "radiation response" terms? That's a great sanity check when adding novel GO concepts.